- **全排列**
  - 链接：全排列
  - 描述

46. 全排列

中等 相关标签 相关企业 A文

给定一个不含重复数字的数组 `nums` ，返回其 *所有可能的全排列* 。你可以 **按任意顺序** 返回答案。

示例 1：

> **输入：** nums = [1,2,3]
> **输出：** [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

示例 2：

> **输入：** nums = [0,1]
> **输出：** [[0,1],[1,0]]

示例 3：

> **输入：** nums = [1]
> **输出：** [[1]]

提示：

- `1 <= nums.length <= 6`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有整数 **互不相同**

  - 答案

```
// 写法1
function permute(nums: number[]): number[][] {
  const resArr: number[][] = []
  const arr: number[] = []
  loop()

  function loop() {
    nums.forEach(num => {
      if (!arr.includes(num)) {
```

```
        arr.push(num)
        if (arr.length === nums.length) {
          resArr.push(arr.slice())
        } else {
          loop()
        }
        arr.pop()
      }
    })
  }

  return resArr
}

// 写法2
function permute(nums: number[]): number[][] {
  const resArr: number[][] = []
  loop()

  function loop(arr: number[] = []) {
    if (arr.length === nums.length) {
      resArr.push(arr)
    } else {
      nums.forEach(num => {
        if (!arr.includes(num)) {
          loop([...arr, num])
        }
      })
    }
  }

  return resArr
}
```

- **子集**
  - 链接：子集
  - 说明

78. 子集

中等 相关标签 相关企业 A文

给你一个整数数组 `nums` ，数组中的元素 **互不相同** 。返回该数组所有可能的子集（幂集）。

解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例 1：

> **输入：** `nums = [1,2,3]`
> **输出：** `[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

示例 2：

> **输入：** `nums = [0]`
> **输出：** `[[],[0]]`

提示：

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有元素 **互不相同**

  - 答案

```
function subsets(nums: number[]): number[][] {
  const result: number[][] = []
  const arr: number[] = []
  loop()

  function loop(index = 0) {
    if (index >= nums.length) {
      result.push(arr.slice())
    } else {
      arr.push(nums[index])
      loop(index + 1) // 有这个数的情况
      arr.pop()
      loop(index + 1) // 没有这个数的情况
    }
```

```
    }

    return result
}
```

- **电话号码的字母组合**
  - 链接：电话号码的字母组合
  - 描述

17. 电话号码的字母组合

中等 相关标签 相关企业 A文

给定一个仅包含数字 `2-9` 的字符串，返回所有它能表示的字母组合。答案可以按 **任意顺序** 返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

示例 1：

> **输入：** `digits = "23"`
> **输出：** `["ad","ae","af","bd","be","bf","cd","ce","cf"]`

示例 2：

> **输入：** `digits = ""`
> **输出：** `[]`

示例 3：

> **输入：** `digits = "2"`
> **输出：** `["a","b","c"]`

提示：

- `0 <= digits.length <= 4`
- `digits[i]` 是范围 `['2', '9']` 的一个数字。

  - 答案

```
const map = {
  '1': '*', // 用任意字符占个位置，保存结果的时候再去掉，以保证 charList.length 的值正确
  '2': 'abc',
  '3': 'def',
  '4': 'ghi',
```

```
  '5': 'jkl',
  '6': 'mno',
  '7': 'pqrs',
  '8': 'tuv',
  '9': 'wxyz',
}

function letterCombinations(digits: string): string[] {
  const result: string[] = []
  const charList: string[] = []

  function loop() {
    for (const char of map[digits.charAt(charList.length) as keyof typeof map]) {
      charList.push(char)
      if (charList.length === digits.length) {
        result.push(charList.filter((item) => item !== '*').join(''))
      } else {
        loop()
      }
      charList.pop()
    }
  }
  digits && loop()

  return result
}
```

- **组合总和**
  - 链接：组合总和
  - 描述

39. 组合总和

已解答

中等 相关标签 相关企业

给你一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target` ，找出 `candidates` 中可以使数字和为目标数 `target` 的 所有 **不同组合** ，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。

`candidates` 中的 **同一个** 数字可以 **无限制重复被选取** 。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 `target` 的不同组合数少于 `150` 个。

示例 1：

> **输入：** candidates = `[2,3,6,7]`, target = `7`
> **输出：** [[2,2,3],[7]]
> **解释：**
> 2 和 3 可以形成一组候选，2 + 2 + 3 = 7 。注意 2 可以使用多次。
> 7 也是一个候选， 7 = 7 。
> 仅有这两种组合。

示例 2：

> **输入：** candidates = [2,3,5], target = 8
> **输出：** [[2,2,2,2],[2,3,3],[3,5]]

示例 3：

> **输入：** candidates = `[2]`, target = 1
> **输出：** []

提示：

- `1 <= candidates.length <= 30`
- `2 <= candidates[i] <= 40`
- `candidates` 的所有元素 **互不相同**
- `1 <= target <= 40`

- 答案

```
function combinationSum(candidates: number[], target: number): number[][] {
  const result: number[][] = []
  const arr: number[] = []
  recursion()

  function recursion(index = 0, sum = 0) {
    for (let i = index; i < candidates.length; i++) {
      const num = candidates[i]
      arr.push(num)
      if (sum + num === target) {
        result.push(arr.slice())
      } else if (sum + num < target) {
        recursion(i, sum + num)
```

```
      }
      arr.pop()
    }
  }

  return result
}
```